

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("Week6 Assignment") \
    .getOrCreate()

d:\Samradni Dahiphale\Projects\Celebal task\
Data_Engineering_Internship_Celebal_Technologies\.venv\Lib\site-
packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not
yet fully support pandas >= 3.0.0. Some features may not work
correctly. It is recommended to use pandas < 3.0.0 for now.
    require_minimum_pandas_version()

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("data/source.csv")

print("Original Data")

df.show()

Original Data
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|product_id|  category|price|stock|old_name|base_price|user_id|
|status|amount|region|priority|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      101|Electronics|55000|  15| Laptop|      55000|  1001|
Completed|  1200| North|   High|
|      102| Furniture|12000|  20|  Chair|      12000|  1002|
Pending|   800| South|   Low|
|      103|Electronics|75000|  10| Mobile|      75000|  NULL|
Completed|  2500| East| Medium|
|      104| Clothing| 2500|  40| Tshirt|       2500|  1004|
Completed|   500| North|   High|
|      105|Electronics|15000|  18| Speaker|      15000|  1005|
Pending|  1800| West|   High|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+

print("Schema")

df.printSchema()

Schema
root
|-- product_id: integer (nullable = true)

```

```
|-- category: string (nullable = true)
|-- price: integer (nullable = true)
|-- stock: integer (nullable = true)
|-- old_name: string (nullable = true)
|-- base_price: integer (nullable = true)
|-- user_id: integer (nullable = true)
|-- status: string (nullable = true)
|-- amount: integer (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)
```

```
print("Select Product ID and Price")
```

```
df.select("product_id", "price").show()
```

```
Select Product ID and Price
```

```
+-----+-----+
|product_id|price|
+-----+-----+
|      101|55000|
|      102|12000|
|      103|75000|
|      104| 2500|
|      105|15000|
+-----+-----+
```

```
#Filter Electronics
```

```
print("Electronics Products")
```

```
df.filter(df.category=="Electronics").show()
```

```
Electronics Products
```

```
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|product_id|category|price|stock|old_name|base_price|user_id|
status|amount|region|priority|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      101|Electronics|55000| 15| Laptop| 55000| 1001|
Completed| 1200| North| High|
|      103|Electronics|75000| 10| Mobile| 75000| NULL|
Completed| 2500| East| Medium|
|      105|Electronics|15000| 18| Speaker| 15000| 1005|
Pending| 1800| West| High|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
```

```
#Rename Column
```

```
df = df.withColumnRenamed("old_name","new_name")
```

```
print("Renamed Column")
```

```
df.show()
```

```
Renamed Column
```

```
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|product_id|  category|price|stock|new_name|base_price|user_id|
status|amount|region|priority|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      101|Electronics|55000|  15|  Laptop|      55000|  1001|
Completed|  1200| North|    High|
|      102|  Furniture|12000|  20|   Chair|      12000|  1002|
Pending|   800| South|    Low|
|      103|Electronics|75000|  10|  Mobile|      75000|   NULL|
Completed|  2500| East|   Medium|
|      104|  Clothing| 2500|  40|  Tshirt|       2500|  1004|
Completed|   500| North|    High|
|      105|Electronics|15000|  18| Speaker|      15000|  1005|
Pending|  1800| West|    High|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
```

```
#Cast Price
```

```
df = df.withColumn(
    "price",
    col("price").cast("double")
)
```

```
print("Price Converted")
```

```
df.printSchema()
```

```
Price Converted
```

```
root
```

```
|-- product_id: integer (nullable = true)
|-- category: string (nullable = true)
|-- price: double (nullable = true)
|-- stock: integer (nullable = true)
|-- new_name: string (nullable = true)
|-- base_price: integer (nullable = true)
|-- user_id: integer (nullable = true)
|-- status: string (nullable = true)
|-- amount: integer (nullable = true)
|-- region: string (nullable = true)
```

```
|-- priority: string (nullable = true)

#Add New Column
df = df.withColumn(
    "final_price",
    col("base_price")*1.18
)

print("Final Price")

df.show()

Final Price
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|product_id| category| price|stock|new_name|base_price|user_id|
status|amount|region|priority|final_price|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|      101|Electronics|55000.0| 15| Laptop| 55000| 1001|
Completed| 1200| North| High| 64900.0|
|      102| Furniture|12000.0| 20| Chair| 12000| 1002|
Pending| 800| South| Low| 14160.0|
|      103|Electronics|75000.0| 10| Mobile| 75000| NULL|
Completed| 2500| East| Medium| 88500.0|
|      104| Clothing| 2500.0| 40| Tshirt| 2500| 1004|
Completed| 500| North| High| 2950.0|
|      105|Electronics|15000.0| 18| Speaker| 15000| 1005|
Pending| 1800| West| High| 17700.0|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+

#Filter Orders
print("Completed Orders")

df.filter(
    (col("status")=="Completed") &
    (col("amount")>1000)
).show()

Completed Orders
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|product_id| category| price|stock|new_name|base_price|user_id|
status|amount|region|priority|final_price|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|      101|Electronics|55000.0| 15| Laptop| 55000| 1001|
Completed| 1200| North| High| 64900.0|
```

103	Electronics	75000.0	10	Mobile	75000	NULL
Completed	2500	East	Medium	88500.0		

#Handle Null Values

`print("Non Null User IDs")`

```
df.filter(
    col("user_id").isNotNull()
).show()
```

Non Null User IDs

product_id	category	price	stock	new_name	base_price	user_id
status	amount	region	priority	final_price		
101	Electronics	55000.0	15	Laptop	55000	1001
Completed	1200	North	High	64900.0		
102	Furniture	12000.0	20	Chair	12000	1002
Pending	800	South	Low	14160.0		
104	Clothing	2500.0	40	Tshirt	2500	1004
Completed	500	North	High	2950.0		
105	Electronics	15000.0	18	Speaker	15000	1005
Pending	1800	West	High	17700.0		

#Filter OR Condition

`print("North OR High")`

```
df.filter(
    (col("region")== "North") |
    (col("priority")== "High")
).show()
```

North OR High

product_id	category	price	stock	new_name	base_price	user_id
status	amount	region	priority	final_price		
101	Electronics	55000.0	15	Laptop	55000	1001
Completed	1200	North	High	64900.0		
104	Clothing	2500.0	40	Tshirt	2500	1004
Completed	500	North	High	2950.0		

105	Electronics	15000.0	18	Speaker	15000	1005
Pending	1800	West	High	17700.0		

#Wide Transformation

```
print("Group By Category")
```

```
df.groupBy("category").count().show()
```

Group By Category

category	count
Electronics	3
Clothing	1
Furniture	1

#Save as CSV

```
df.write \
.mode("overwrite") \
.option("header", True) \
.csv("output/csv_output")
```

#Save as Parquet

```
df.write \
.mode("overwrite") \
.parquet("output/parquet_output")
```

#Read Parquet

```
parquet_df = spark.read.parquet("output/parquet_output")
```

```
print("Reading Parquet")
```

```
parquet_df.show()
```

Reading Parquet

product_id	category	price	stock	new_name	base_price	user_id
status	amount	region	priority	final_price		
101	Electronics	55000.0	15	Laptop	55000	1001
Completed	1200	North	High	64900.0		
102	Furniture	12000.0	20	Chair	12000	1002
Pending	800	South	Low	14160.0		
103	Electronics	75000.0	10	Mobile	75000	NULL

```
Completed| 2500| East| Medium| 88500.0|
| 104| Clothing| 2500.0| 40| Tshirt| 2500| 1004|
Completed| 500| North| High| 2950.0|
| 105| Electronics| 15000.0| 18| Speaker| 15000| 1005|
Pending| 1800| West| High| 17700.0|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
```

```
#Stop Spark
spark.stop()
```

Apache Spark – Week 6 Assignment

Brief Insights on Performance and Architecture

Spark Architecture

Apache Spark follows a distributed computing architecture consisting of three main components:

- **Driver:** The central controller that creates the `SparkSession`, builds the Directed Acyclic Graph (DAG), schedules tasks, and collects results from executors.
- **Cluster Manager:** Allocates computing resources (CPU and memory) to Spark applications. Common cluster managers include Standalone, YARN, Mesos, and Kubernetes.
- **Executors:** Worker processes that execute tasks assigned by the Driver, perform computations on data partitions, cache intermediate results, and return the output.

This architecture enables Spark to process large datasets efficiently across multiple machines.

Performance Insights

1. Lazy Evaluation

Spark delays execution of transformations until an action such as `show()` or `count()` is called. During this time, Spark builds a Directed Acyclic Graph (DAG) and optimizes the execution plan, reducing unnecessary computations and improving overall performance.

2. Transformations and Actions

Operations such as `select()`, `filter()`, `withColumn()`, and `withColumnRenamed()` are transformations and are evaluated lazily. Actions such as `show()` trigger the execution of all pending transformations.

3. Schema Inference

Using `inferSchema=True` automatically detects appropriate data types for each column, eliminating the need for manual schema definition and enabling more efficient processing.

4. Data Filtering

Filtering records early in the processing pipeline reduces the amount of data that must be processed by subsequent operations, improving execution speed and lowering memory usage.

5. Data Type Conversion

Casting the `price` column from Integer/String to Double ensures accurate mathematical calculations and better compatibility with analytical operations.

6. Handling Null Values

Removing or filtering null values improves data quality and prevents errors during analysis and aggregation.

7. Wide Transformations

Operations such as `groupBy()` require data to be shuffled across executors. Shuffle operations are computationally expensive; therefore, they should be minimized whenever possible to achieve better performance.

8. CSV vs Parquet

- **CSV** is a row-based text format that occupies more storage space and requires more time to read.
- **Parquet** is a columnar storage format that supports compression and Predicate Pushdown, allowing Spark to read only the required columns and rows.

For analytical workloads, Parquet provides significantly better performance and reduced storage requirements.

9. Efficient Data Pipeline

The implemented Spark pipeline follows the sequence:

Read CSV → Apply Transformations → Filter Records → Handle Null Values → Save as CSV → Save as Parquet → Read Parquet

This workflow demonstrates a typical Extract-Transform-Load (ETL) process in Apache Spark.

10. Best Practices

- Use `show()` for inspecting data instead of `collect()` on large datasets.
 - Apply filters as early as possible.
 - Prefer Parquet over CSV for analytical processing.
 - Minimize shuffle operations.
 - Use appropriate data types for computations.
 - Stop the Spark session after execution to release resources.
-

Conclusion

This assignment demonstrates the fundamental concepts of Apache Spark, including distributed architecture, lazy evaluation, DataFrame transformations, schema handling, filtering, null value management, and file format optimization. The implementation shows how Spark efficiently processes large datasets through parallel execution and optimized query planning while following industry best practices for scalable data processing.

Apache Spark – Week 6 Assignment

Objective

The objective of this assignment is to understand Apache Spark architecture and perform efficient data processing using DataFrame operations, schema handling, filtering, transformations, optimized file formats, and performance optimization techniques.

Week-6 Questions & Answers

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer

Driver

- The Driver is the main program of a Spark application.

- It creates the SparkSession/SparkContext.
- Builds the DAG (Directed Acyclic Graph).
- Schedules tasks for execution.
- Collects results from executors.

Cluster Manager

- Manages cluster resources such as CPU and memory.
- Allocates resources to Spark applications.
- Examples include Standalone, YARN, Mesos, and Kubernetes.

Executor

- Executors run on worker nodes.
 - Execute tasks assigned by the Driver.
 - Cache data in memory.
 - Return results back to the Driver.
-

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer

Spark does not execute transformations immediately.

Instead, it:

- Records all transformations.
- Builds a Directed Acyclic Graph (DAG).
- Optimizes the execution plan.
- Executes only when an Action (such as `show()` or `count()`) is called.

Benefits

- Reduces unnecessary computations.
 - Optimizes execution.
 - Minimizes disk I/O.
 - Improves performance.
-

Q3. Write a Spark command to read a CSV file located at `data/source.csv`, ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read \  
    .option("header", True) \  
    .option("inferSchema", True) \  
    .csv("data/source.csv")
```

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer

CSV	Parquet
Row-based storage	Columnar storage
Larger file size	Smaller compressed files
Slower reading	Faster reading
No schema information	Stores schema
Reads entire rows	Reads only required columns

Why it matters

Parquet offers better performance because:

- Reads only required columns.
- Supports Predicate Pushdown.
- Uses compression.
- Reduces disk I/O.

Q5. Given a DataFrame `df`, write a query to select the columns `product_id` and `price` where the category is `'Electronics'`.

```
df.filter(df.category == "Electronics") \
    .select("product_id", "price") \
    .show()
```

Q6. Write the code to rename the column `old_name` to `new_name` and cast the `price` column from String to Double.

```
from pyspark.sql.functions import col

df = df.withColumnRenamed("old_name", "new_name")

df = df.withColumn(
    "price",
    col("price").cast("double")
)
```

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer

Spark stores all transformations in a Lineage Graph (DAG).

If a worker node fails:

- Spark identifies the lost partitions.
- Recomputes only those partitions using the lineage.

- Does not recompute the entire dataset.

This provides efficient fault tolerance without requiring multiple copies of intermediate data.

Q8. Write a query to filter a DataFrame `df_orders` for rows where the status is `'Completed'` AND the amount is greater than 1000.

```
df_orders.filter(  
    (df_orders.status == "Completed") &  
    (df_orders.amount > 1000)  
) .show()
```

Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer

Predicate Pushdown allows Spark to push filter conditions directly to the Parquet reader.

Instead of reading the entire dataset:

- Spark reads only matching rows.
- Reads only required columns.

Benefits:

- Less memory usage.
 - Less disk I/O.
 - Faster query execution.
 - Better overall performance.
-

Q10. Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col

df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)
```

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Answer

Transformations

Transformations create a new DataFrame or RDD but are executed lazily.

Examples:

- `filter()`
- `select()`

Actions

Actions trigger execution and return results.

Examples:

- `show()`
 - `count()`
-

Q12. Write the Spark command to load a Parquet file from `path/to/input`, filter out any rows where `user_id` is null, and save the result as a CSV at `path/to/output`.

```
df = spark.read.parquet("path/to/input")

df.filter(df.user_id.isNotNull()) \
    .write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer

Client Mode	Cluster Mode
Driver runs on the local machine	Driver runs inside the cluster
Client must remain connected	Client can disconnect after job submission
Mainly used for development	Mainly used for production
Lower fault tolerance	Higher fault tolerance

Q14. Write a query to filter a dataset for rows where the region is `'North'` OR the priority is `'High'`.

```
df.filter(
    (df.region == "North") |
```



```
(df.priority == "High")  
) .show()
```

Q15. When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer

`show(5)` displays only the first five rows, making it suitable for inspecting large datasets.

`collect()` retrieves the entire dataset from all executors to the Driver's memory.

Using `collect()` on a multi-terabyte dataset may:

- Cause `OutOfMemoryError`.
- Slow down execution.
- Crash the Driver.

Therefore, `show(5)` is the recommended approach for exploring large datasets.

Conclusion

This assignment demonstrates the core concepts of Apache Spark, including Spark Architecture, Lazy Evaluation, DAG execution, DataFrame transformations, schema handling, filtering, null value handling, wide transformations, and optimized file formats. It also highlights best practices such as using Parquet for analytics, minimizing shuffle operations, applying filters early, and using `show()` instead of `collect()` to efficiently process large-scale datasets.